AI Assisted Coding Assignment- 9.5

2303A51886 : 30 SAI SPURTHI. B 20.02.2026

Lab Experiment: Documentation Generation -Automatic documentation and code comments.

Lab Objectives

1. To understand automatic documentation generation.
2. To generate code comments and docstrings using AI tools.
3. To learn the importance of documentation in software development.
   Lab Outcomes
4. Students will be able to generate documentation automatically for code.
5. Students will be able to add clear comments and docstrings to programs.
6. Students will be able to improve code readability and maintainability using documentation.

Problem 1: String Utilities Function
Consider the following Python function:
def reverse_string(text):
return text[::-1]
Task:

1. Write documentation in:
   o (a) Docstring
   o (b) Inline comments
   o (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

Problem 2: Password Strength Checker
Consider the function:
def check_strength(password):
return len(password) >= 8
Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

Problem 3: Math Utilities Module
Task:

1. Create a module math_utils.py with functions:
   o square(n)
   o cube(n)
   o factorial(n)
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

Problem 4: Attendance Management Module
Task:

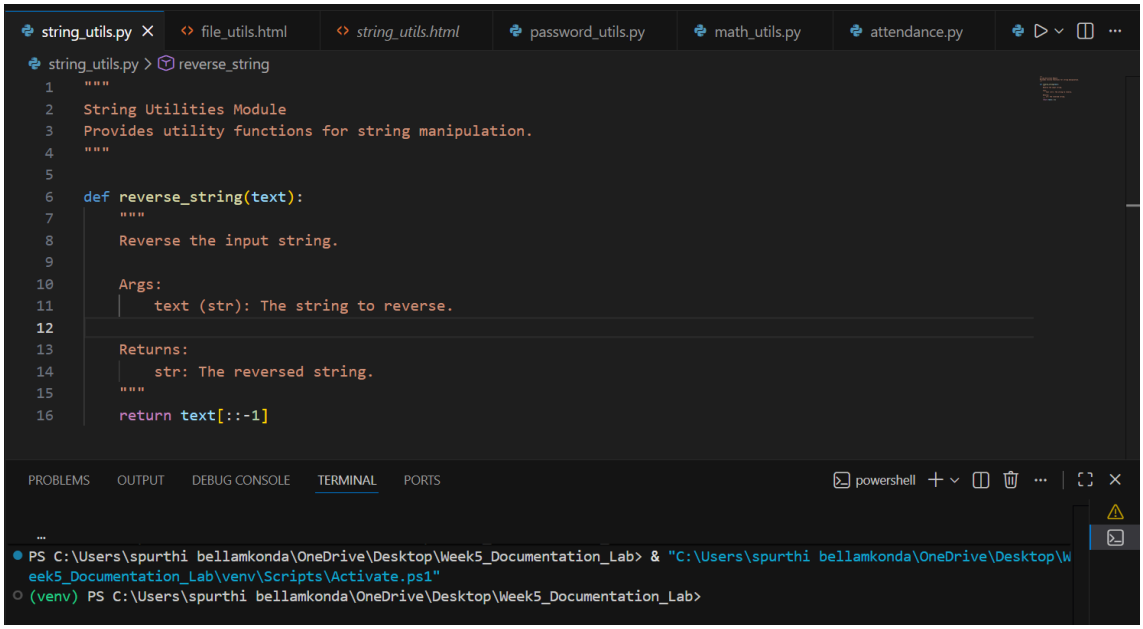
1. Create a module attendance.py with functions:
   o mark_present(student)
   o mark_absent(student)
   o get_attendance(student)
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

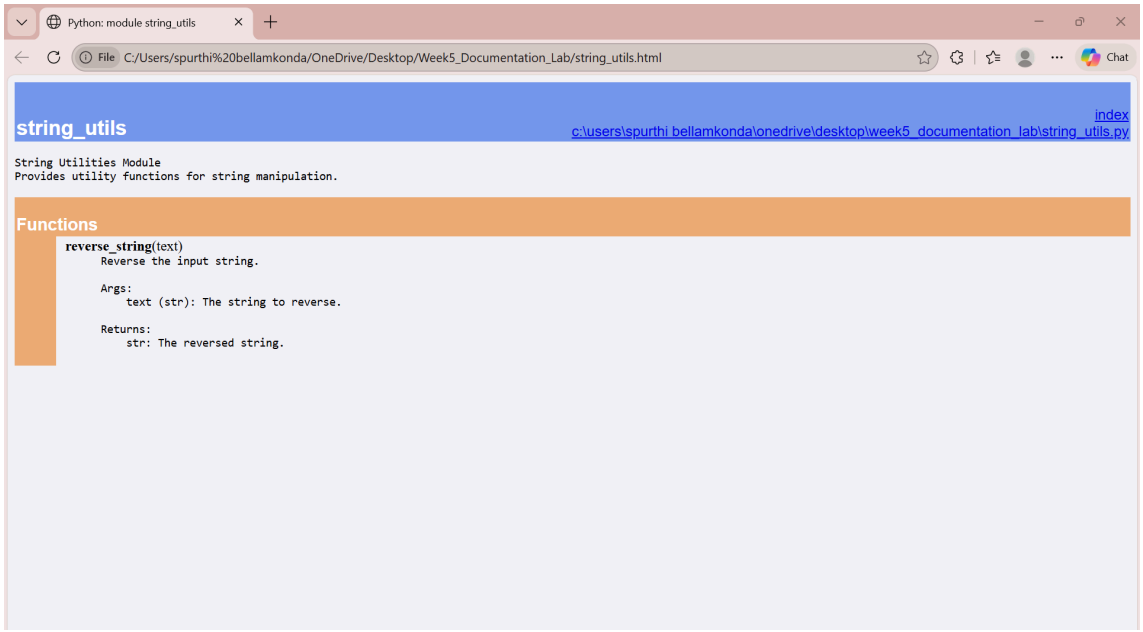
Problem 5 : File Handling Function
Consider the function:

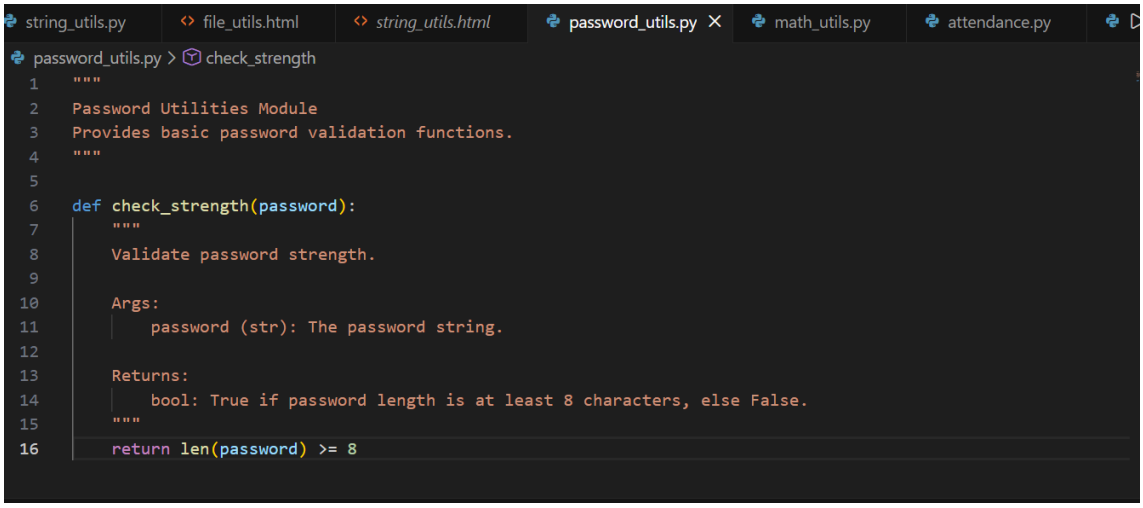
```
def read_file(filename):
with open(filename, 'r') as f:
return f.read()
```

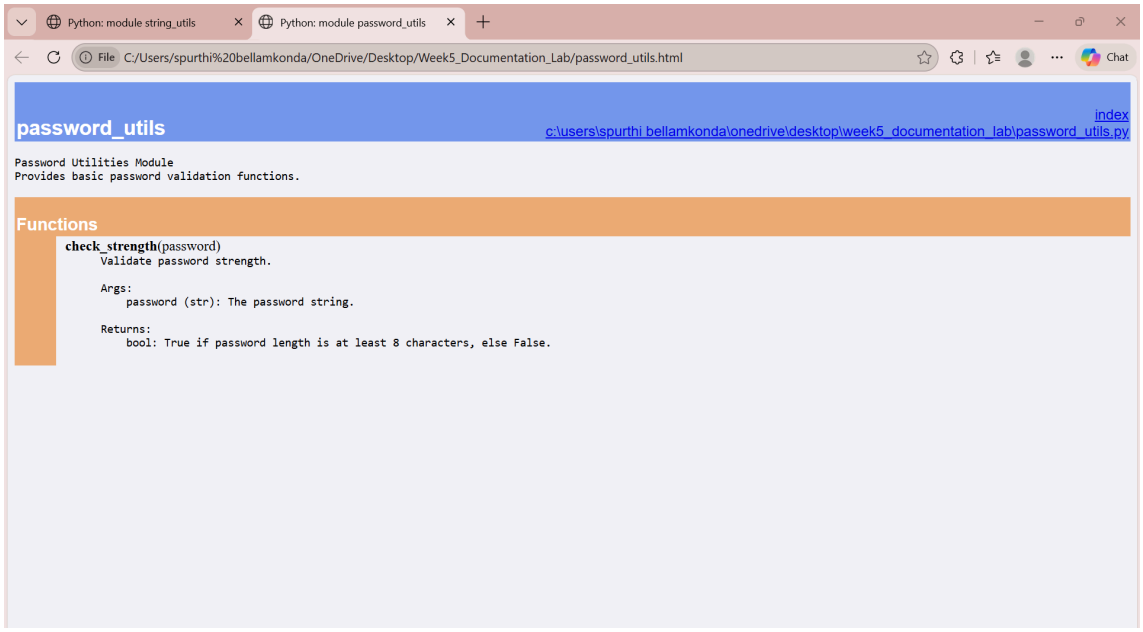
Task:

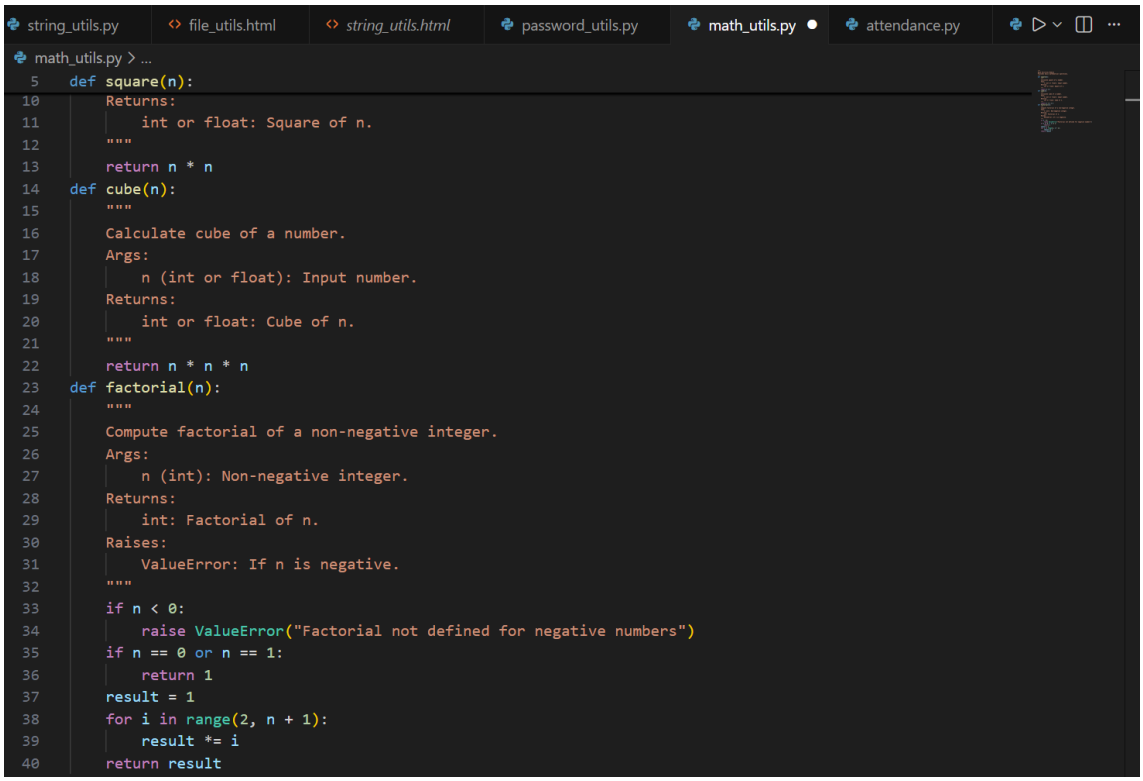
1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

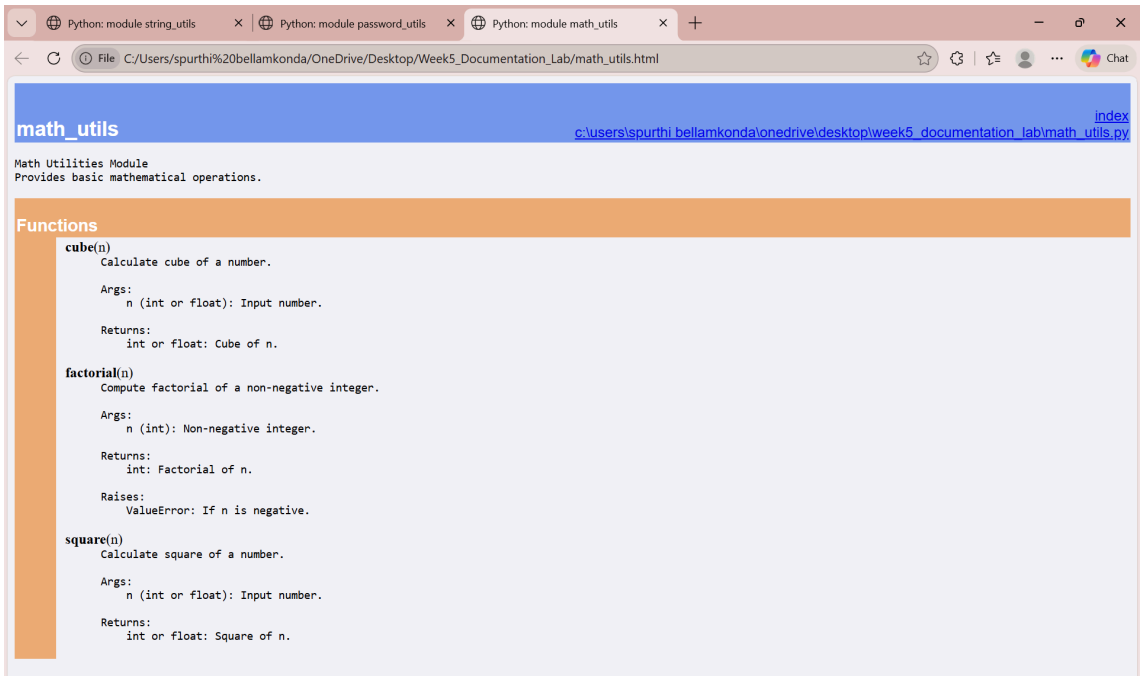

string_utils.py
file_utils.html
string_utils.html
password_utils.py
math_utils.py
attendance.py
string_utils.py > reverse_string
"""
String Utilities Module
Provides utility functions for string manipulation.
"""
def reverse_string(text):
"""
Reverse the input string.
Args:
text (str): The string to reverse.
Returns:
str: The reversed string.
"""
return text[::-1]
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
powershell
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\Week5_Documentation_Lab> & "C:\Users\spurthi bellamkonda\OneDrive\Desktop\Week5_Documentation_Lab\venv\Scripts\Activate.ps1"
(venv) PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\Week5_Documentation_Lab>

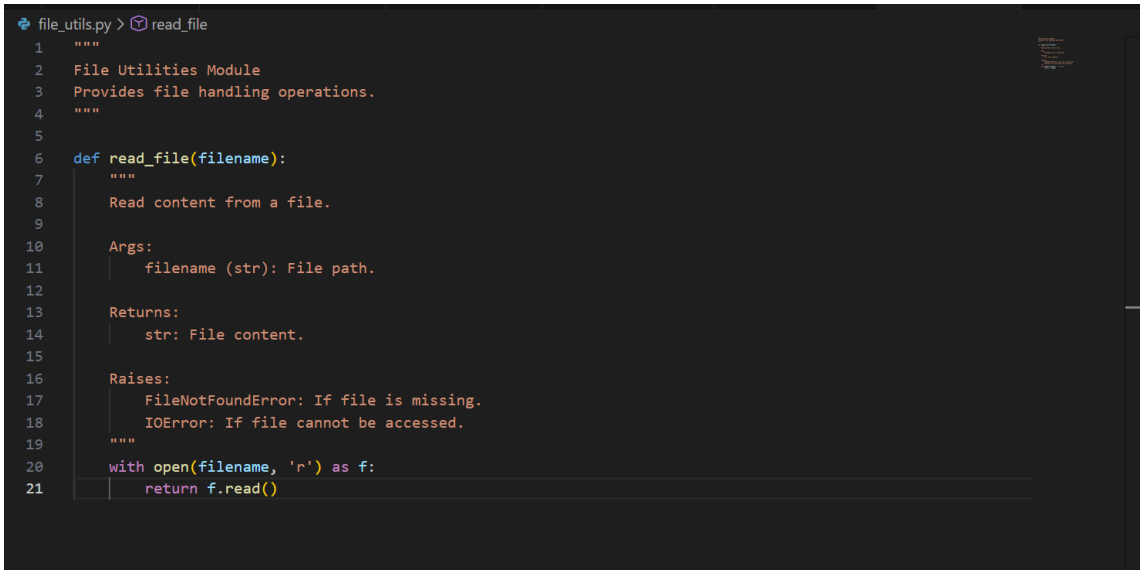

Python: module string_utils
File C:/Users/spurthi%20bellamkonda/OneDrive/Desktop/Week5_Documentation_Lab/string_utils.html
Chat
string_utils
index
c:\users\spurthi bellamkonda\onedrive\desktop\week5_documentation_lab\string_utils.py
String Utilities Module
Provides utility functions for string manipulation.
Functions
reverse_string(text)
Reverse the input string.
Args:
text (str): The string to reverse.
Returns:
str: The reversed string.


string_utils.py
file_utils.html
string_utils.html
password_utils.py
math_utils.py
attendance.py
password_utils.py > check_strength
"""
Password Utilities Module
Provides basic password validation functions.
"""
def check_strength(password):
"""
Validate password strength.
Args:
password (str): The password string.
Returns:
bool: True if password length is at least 8 characters, else False.
"""
return len(password) >= 8


Python: module string_utils
Python: module password_utils
File C:/Users/spurthi%20bellamkonda/OneDrive/Desktop/Week5_Documentation_Lab/password_utils.html
Chat
password_utils
index
c:\users\spurthi bellamkonda\onedrive\desktop\week5_documentation_lab\password_utils.py
Password Utilities Module
Provides basic password validation functions.
Functions
check_strength(password)
Validate password strength.
Args:
password (str): The password string.
Returns:
bool: True if password length is at least 8 characters, else False.


string_utils.py
file_utils.html
string_utils.html
password_utils.py
math_utils.py
attendance.py
math_utils.py > ...
def square(n):
Returns:
int or float: Square of n.
"""
return n * n
def cube(n):
"""
Calculate cube of a number.
Args:
n (int or float): Input number.
Returns:
int or float: Cube of n.
"""
return n * n * n
def factorial(n):
"""
Compute factorial of a non-negative integer.
Args:
n (int): Non-negative integer.
Returns:
int: Factorial of n.
Raises:
ValueError: If n is negative.
"""
if n < 0:
raise ValueError("Factorial not defined for negative numbers")
if n == 0 or n == 1:
return 1
result = 1
for i in range(2, n + 1):
result *= i
return result


Python: module string_utils
Python: module password_utils
Python: module math_utils
File C:/Users/spurthi%20bellamkonda/OneDrive/Desktop/Week5_Documentation_Lab/math_utils.html
Chat
math_utils
index
c:\users\spurthi bellamkonda\onedrive\desktop\week5_documentation_lab\math_utils.py
Math Utilities Module
Provides basic mathematical operations.
Functions
cube(n)
Calculate cube of a number.
Args:
n (int or float): Input number.
Returns:
int or float: Cube of n.
factorial(n)
Compute factorial of a non-negative integer.
Args:
n (int): Non-negative integer.
Returns:
int: Factorial of n.
Raises:
ValueError: If n is negative.
square(n)
Calculate square of a number.
Args:
n (int or float): Input number.
Returns:
int or float: Square of n.



file_utils

index
c:\users\spurthi bellamkonda\onedrive\desktop\week5_documentation_lab\file_utils.py

File Utilities Module
Provides file handling operations.

Functions

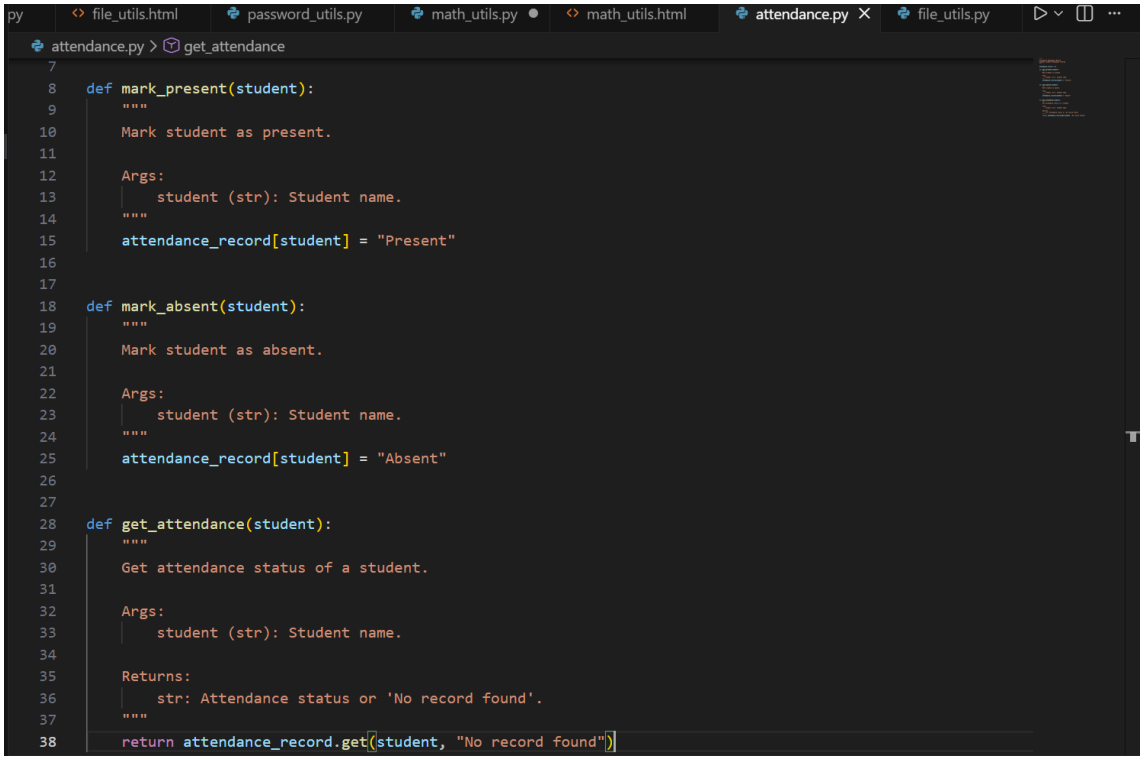
read_file(filename)

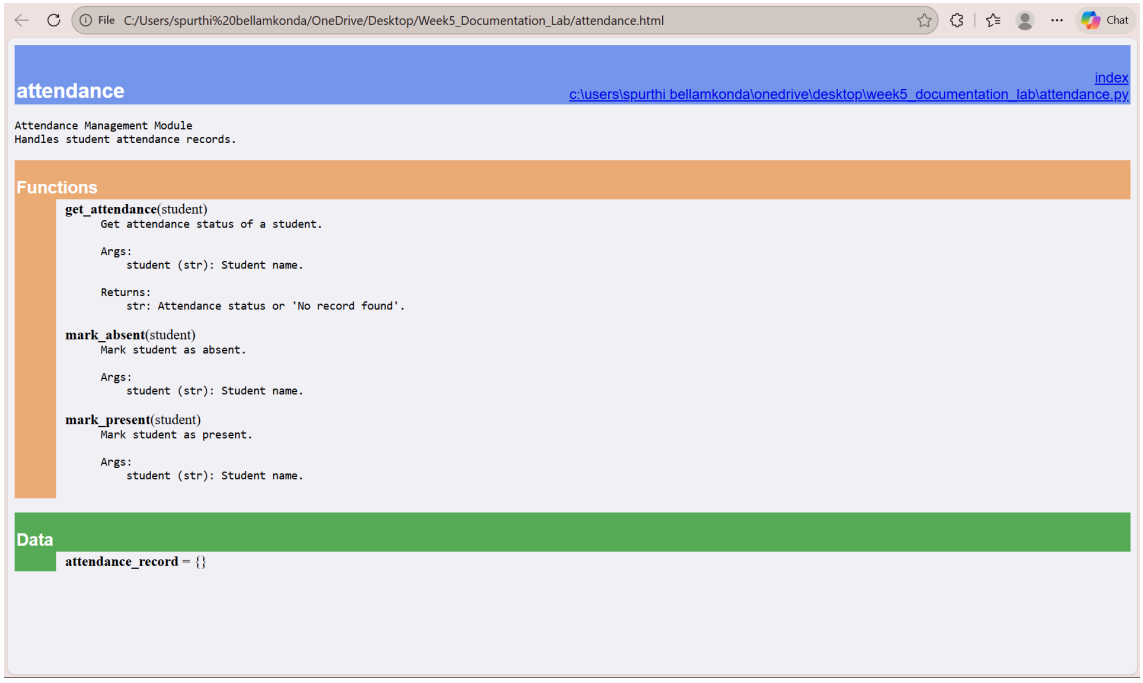
```
Read content from a file.

Args:
    filename (str): File path.

Returns:
    str: File content.

Raises:
    FileNotFoundError: If file is missing.
    IOError: If file cannot be accessed.
```


file_utils.html
password_utils.py
math_utils.py
math_utils.html
attendance.py
file_utils.py
attendance.py > get_attendance
def mark_present(student):
"""
Mark student as present.
Args:
student (str): Student name.
"""
attendance_record[student] = "Present"
def mark_absent(student):
"""
Mark student as absent.
Args:
student (str): Student name.
"""
attendance_record[student] = "Absent"
def get_attendance(student):
"""
Get attendance status of a student.
Args:
student (str): Student name.
Returns:
str: Attendance status or 'No record found'.
"""
return attendance_record.get(student, "No record found")


File C:/Users/spurthi%20bellamkonda/OneDrive/Desktop/Week5_Documentation_Lab/attendance.html
Chat
attendance
index
c:\users\spurthi bellamkonda\onedrive\desktop\week5_documentation_lab\attendance.py
Attendance Management Module
Handles student attendance records.
Functions
get_attendance(student)
Get attendance status of a student.
Args:
student (str): Student name.
Returns:
str: Attendance status or 'No record found'.
mark_absent(student)
Mark student as absent.
Args:
student (str): Student name.
mark_present(student)
Mark student as present.
Args:
student (str): Student name.
Data
attendance_record = {}